# Fundamentals

## MOS Transistors

**Def. 1.** ***n-type***: *Closed (ON) when Gate is High (1). Excellent at passing 0 (strong 0).*

**Def. 2.** ***p-type***: *Closed (ON) when Gate is Low (0). Excellent at passing 1 (strong 1).*

**Def. 3.** ***CMOS***: *Organized into two networks to prevent static power draw:*

- ***Pull-Up (PUN)***: *Built with pMOS to connect output to $V_{DD}$ (1).*
- ***Pull-Down (PDN)***: *Built with nMOS to connect output to Ground (0).*

## Logic Gates

**Def. 4.** ***NOT***: *2 transistors (1 pMOS, 1 nMOS).*

$$Y = \overline{A}$$

***NAND***: *4 transistors.*

$$Y = \overline{A \cdot B}$$

***NOR***: *4 transistors.*

$$Y = \overline{A + B}$$

***XOR***: *(1 if different)*

$$Y = A \oplus B$$

***Buffer***: *(two NOT gates)*

$$Y = A$$

**Imp. 5.** *A set of gates is* ***logically complete*** *if any Boolean function can be realized using only those gates (e.g. $\{NAND\}$, $\{NOR\}$).*

## Boolean Algebra

**Thm. 6.** ***DeMorgan's theorems***:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

**Def. 7.** ***Minterm***: *AND term containing all input variables.* ***Sum of Products (SOP)***: *OR of minterms for which output is 1.*

## Algorithm 8

```
#Karnaugh Map (K-Map) format
1. Map truth table into grid where adjacent cells differ by 1 bit (Gray code).
2. Circle groups of 2^n adjacent 1s.
3. Eliminate variables that change within the circle.
```

# Combinational Logic

**Def. 9.** *Combinational Logic output is a pure function of current inputs (no memory).*

## Building Blocks

**Def. 10.** *An n-to-$2^n$* ***decoder*** *sets exactly one of its $2^n$ outputs to 1 based on its n-bit input.*

**Def. 11.** *A* ***Multiplexer (MUX)*** *uses $log_2 N$ select lines to route one of $N$ data inputs to the output.*

**Def. 12.** *A* ***Full Adder***:

$$\text{Sum: } S = A \oplus B \oplus C_{in}$$

$$\text{Carry: } C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

**Imp. 13.** *An n-bit* ***Ripple Carry Adder (RCA)*** *chains n Full Adders. The critical path is the carry "ripple" from LSB to MSB.*

**Lem. 14.** *A* ***Tri-state buffer*** *has states: $0, 1$, and* ***High-Z*** *(disconnected, allowing multiple components to share a bus).*

**Ex. 15.** ***Programmable Logic Array (PLA)***: *Programmable AND-plane and OR-plane representing a Sum-of-Products.*

# Sequential Logic

**Def. 16.** *Sequential logic incorporates storage elements to maintain "state", producing outputs based on both current and past inputs.*

## Fundamental Blocks

**Def. 17.** ***Reset-Set (R-S) latch*** *(built from cross-coupled NAND gates): Set ($S = 0$): $Q = 1$, Reset ($R = 0$): $Q = 0$. $S = 0, R = 0$ is forbidden (metastability).*

**Def. 18.** ***Gated D Latch*** *adds Write Enable ($WE$). Transparent when $WE = 1$ (output $Q$ follows $D$), holds value otherwise.*

**Def. 19.** ***D Flip-Flop*** *uses two D latches (master-slave) to be* ***edge-triggered****. Captures $D$ value exactly at rising edge of clock ($0 \to 1$).*

## Timing and Finite State Machines

**Def. 20.** ***Clock (CLK)***: *Periodic signal coordinating synchronous state transitions. Period must exceed the worst-case combinational delay.*

**Def. 21.** ***Finite State Machine (FSM)*** *has:*

- ***Moore FSM***: *Outputs depend* ***only*** *on current state.*
- ***Mealy FSM***: *Outputs depend on current state* ***and*** *current inputs.*

**Rem. 22.** *FSM Implementation: 1. State Register (sequential) 2. Next State Logic (combinational) 3. Output Logic (combinational)*

**Ex. 23.** *State Encodings:* ***Binary*** *(min bits),* ***One-Hot*** *(1 bit/state, simple logic),* ***Output*** *(state bits = output bits).*

**Def. 24.** ***Register***: *Group of D flip-flops with common clock/enable.* ***Memory***: *Storage locations indexed by address.*

# Timing

## Combinational Timing

**Def. 25.** ***Propagation Delay*** *($t_{pd}$): Maximum time from input change until output reaches final value.* ***Contamination Delay*** *($t_{cd}$): Minimum time from input change until output starts to change.*

**Ex. 26.** ***Glitch***: *Single input transition causing multiple output transitions due to different path delays. Fix with redundant "consensus terms" in K-maps.*

## Sequential Timing

**Def. 27.** ***Aperture time*** *($t_a = t_{setup} + t_{hold}$):*

- ***Setup Time*** *($t_{setup}$): Time before clock edge data must be stable.*
- ***Hold Time*** *($t_{hold}$): Time after clock edge data must remain stable.*

*Violations lead to* ***metastable*** *states.*

**Def. 28.** ***Prop. Clock-to-Q Delay*** *($t_{pcq}$): Max time after clock edge until $Q$ is stable.* ***Cont. Clock-to-Q Delay*** *($t_{ccq}$): Min time after clock edge until $Q$ starts to change.*

**Thm. 29.** *For a synchronous system to operate correctly:* ***Setup Constraint (Max Freq)***:

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

***Hold Constraint***:

$$t_{ccq} + t_{cd} > t_{hold}$$

## Clock Skew

**Def. 30.** ***Clock skew*** *($t_{skew}$): Time difference between arrival of clock signal at different flip-flops. Worsens both constraints:*

- ***Setup***: $T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$
- ***Hold***: $t_{ccq} + t_{cd} > t_{hold} + t_{skew}$

# Von Neumann Model and ISA

**Def. 31.** *The* ***Von Neumann model*** *consists of memory (stores program and data), processing unit (ALU, registers), control unit (PC, IR), input, and output. Key properties:* ***Stored Program*** *and* ***Sequential Processing****.*

**Rem. 32.** *Instruction Cycle: Fetch → Decode → Evaluate Address → Fetch Operands → Execute → Store Result.*

**Def. 33.** ***Address Space***: *Total unique memory locations.* ***Addressability***: *Bits stored per address.* ***Endianness***: *Order of bytes (Big Endian stores MSB at lowest address).*

## Instruction Set Architecture (ISA)

**Def. 34.** *The* ***ISA*** *is the interface between software and hardware. Defines memory organization, register set, and instructions (Opcode + Operands).*

**Lem. 35.** ***CISC***: *Complex instructions, small semantic gap.* ***RISC***: *Simple instructions, large semantic gap (e.g. MIPS, LC-3).*

**Def. 36.** ***Addressing Modes***: *Immediate, Register, PC-Relative ($PC + offset$), Indirect, Base + Offset.*

## Assembly

**Def. 37.** *Assembly is human-readable machine code. In RISC, operands must be in registers for ALUs:*

- ***Load**: Reads data from memory into a register.*
- ***Store**: Writes data from a register into memory.*

**Rem. 38.** *Execution order altered by **Conditional Branches** (uses condition codes or register comparisons) and **Unconditional Jumps**.*

## Microarchitecture

**Def. 39.** ***Microarchitecture** implements an ISA. One ISA can have many microarchitectures (single-cycle, multi-cycle, pipelined) optimized for performance, cost, or power.*

### Single-Cycle Microarchitecture

**Def. 40.** *Every instruction takes exactly one clock cycle.*

- ***CPI**: Always 1.*
- ***Clock Cycle Time** ($T_c$): Bounded by the slowest instruction (usually `lw`).*

### Multi-Cycle Microarchitecture

**Def. 41.** *Breaks instruction execution into multiple shorter cycles. Cycles take variable steps. Allows **hardware reuse** (e.g. ALU) and requires internal microarchitectural registers (IR, MDR, ALUOut).*

**Ex. 42.** *Phases: 1. Fetch, 2. Decode, 3. Execute, 4. Memory, 5. Write-back.*

**Thm. 43.**

$$Time = \#instr \times Average_CPI \times T_c$$

*Multi-cycle has higher CPI but shorter $T_c$, improving performance over single-cycle. Downside: **Sequencing overhead** added at each cycle transition.*

**Rem. 44.** *Multi-cycle control unit is implemented as an FSM to track instruction steps.*

## Pipelining

**Def. 45.** ***Pipelining** overlaps multiple instructions in execution (assembly line logic). Increases **instruction throughput** but does not decrease latency.*

**Thm. 46.** *A k-stage pipeline ideal speedup is k-fold, but limited by unbalanced stages, sequencing overhead, and hazards.*

### The 5-Stage Pipeline

**Def. 47.** *Stages (MIPS): 1. Fetch (IF), 2. Decode (ID), 3. Execute (EX), 4. Memory (MEM), 5. Write-Back (WB). **Pipeline Registers** (e.g. IF/ID) pass data and control signals forward between stages.*

### Hazards

**Def. 48.** *A hazard prevents the next instruction from executing normally:*

- ***Structural**: Conflict for hardware resources.*
- ***Data**: Instruction depends on data from a non-completed instruction.*
- ***Control**: Branch outcome delays fetching correct next instruction.*

**Def. 49.** *Data dependencies: **Read-After-Write (RAW)**: Instruction needs data about to be written. (Only hazard needing hardware fix in 5-stage pipeline).*

### Resolving Hazards

**Lem. 50.** ***Forwarding (Bypassing)** routes result from EX or MEM directly to ALU input.*

**Imp. 51.** ***Load-Use Hazard** cannot be forwarded. Requires a **Stall** (freeze PC, insert pipeline bubble).*

**Rem. 52.** ***Control Hazards** force a pipeline flush on misprediction. Strategies: Static prediction, Dynamic prediction (Branch History Table), Early Resolution (moving compare to Decode).*

### Advanced Issues

**Def. 53.** ***Precise Exceptions**: Must stop execution on exception instruction while ensuring prior instructions finish correctly. **Out-of-Order**: Multi-cycle functional units cause out-of-order completion. **Fine-Grained Multithreading**: Switches thread every clock to hide hazards.*